

depSNORT Tool Assessment

Independent Evaluation from Operational Use

Project	iHBV-TM-022
Author	HoneyBadger Vanguard LLC
Date	2026-08-14
Classification	Internal — Purple Team Research

depSNORT Tool Assessment

Program: TM-022 Purple Validation Harness — Wave 1 (adjunct) **Project code:** iHBV-TM-022 **Subject:** MoSLoF/depSNORT @ 5b02377 **Assessor:** HoneyBadger Vanguard LLC **Date:** 14 August 2026
Distribution: Internal — Purple Team Research

Executive Summary

depSNORT is a static supply-chain analyser positioned as "an IDS for the dependency supply chain": it parses lockfiles and manifests across six package ecosystems and matches the resulting dependency graph against a modular pack of vector checks — **without ever executing a package manager or firing an install hook**. This assessment is drawn from operational use of the tool against `impacket` in Wave 1 of the TM-022 harness; it is not a code review of depSNORT's implementation but an evaluation of the tool as a product from the perspective of a security consumer.

Bottom line

depSNORT **does what it claims and is safe to hand a red or purple team in its current form**. The three most consequential design choices — zero third-party dependencies, refusal to speculatively resolve unpinned specifiers (D-01), and clean separation of gate-eligible from advisory findings via an exit-code contract — each pay off in observable ways during real use. Two areas warrant improvement before wider adoption: the bundled offline OSV dataset is scoped narrowly enough that a policy-restricted CI runner gets less real coverage than the README implies, and the current version-string handling for source builds is misleading enough to burn an operator once.

Dimension	Grade	Notes
Correctness of findings on impacket	A	One real finding (pyasn1 dormancy), zero false positives, correct null result on <code>setuptools's cmdclass.install</code>
Precision (false-positive rate)	A	The "named credentials only, not blanket env reads" design (VC-002c) demonstrably held
Coverage (breadth)	B+	Six ecosystems, five distinct vector-check families; no <code>poetry.lock</code> / <code>uv.lock</code> yet
Coverage (depth)	B	Bundled offline OSV tier is narrower than the README's air-gapped narrative suggests
Operational fit (CI, gating)	A	Exit-code contract is disciplined, <code>-fail-on-incomplete</code> behaved as advertised
Reporting	A-	JSON, SARIF, DOT, PDF all worked first-try; PDF is spartan but reproducible
Dogfooding	A	<code>sbom</code> really does show <code>components: []</code> — the claim is self-verifying
Documentation	A	The README is unusually thorough; design decisions are enumerated
Version identification (source build)	C	<code>depsnort version</code> reports <code>dev</code> for source builds; the README's "check what build you have" advice does not apply

1. Tool overview

`depSNORT` is a single static Go binary (`~10 MB` , pure `stdlib` , static build with `CGO_ENABLED=0`). It statically resolves the dependency graph of a target project across six ecosystems (npm, PyPI, RubyGems, Cargo, Composer, NuGet) and runs a modular pack of vector-check plugins over that graph. It is explicitly positioned against, and complementary to, tools that watch for outdated packages (Dependabot, Renovate) — its concern is the *composition and shape* of the tree, not staleness.

1.1 The vector-check pack

ID	Axis	Severity	Gate class	Description
VC-001	known-compromise	critical	block	<code>package@version</code> in a known-malicious advisory (<code>MAL-*</code>)
VC-002a	known-compromise	low	gate-eligible	Package declares an install lifecycle hook
VC-002b	known-compromise	medium	gate-eligible	Install hook reaches the network
VC-002c	known-compromise	high	gate-eligible	Install hook references named credentials
VC-002d	known-compromise	critical	block	Hook is exfil-capable (creds + network)
VC-002e	known-compromise	high	gate-eligible	Install hook decodes and executes code
VC-002f	known-compromise	critical	block	Install hook fetches and executes remote code (download cradle)
VC-003	known-compromise	critical	block	Package matches an entry in the operator's IOC ledger
VC-004	weather	medium	advisory	Version published after prolonged dormancy
VC-005	weather	medium	advisory	Pinned version arrived in an anomalous release burst
VC-006	weather	medium	advisory	Package name is a near-miss (typosquat) of a popular package
VC-007	weather	high	gate-eligible	Internal-looking package resolves from a public registry (dep-confusion)
VC-008	vuln	medium	advisory	Package has a disclosed CVE

1.2 The exit-code contract

Code	Meaning
0	Clean, or only advisory findings
1	A block-class finding was present
2	A gate-eligible finding was present and <code>-fail-on-eligible</code> was set
3	Coverage was degraded and <code>-fail-on-incomplete</code> was set
64	Usage error
70	Internal / operational error

The critical structural guarantee is that **advisory findings never change the exit code**, regardless of policy — this was verified during use.

2. Test setup

- Cloned MoSLoF/depSNORT at commit 5b02377 from a fresh worktree.
- Built with `CGO_ENABLED=0 go build -o depsnort ./cmd/depsnort` (Go 1.24.7). Build was clean in one command with no external module fetches, as advertised.
- Verified the dogfooding claim: `./depsnort sbom` produced a CycloneDX document whose `components` array is empty (parsed and asserted with `python3 -c "import json,sys; assert len(json.load(sys.stdin).get('components',[])) == 0"`).
- Ran against `impacket` (Python / PyPI target) in three modes:
 1. Default scan of the repository as-shipped (unpinned).
 2. `-fail-on-incomplete` for CI-gate simulation.
 3. Scan of a machine-generated pin set built from an installed venv.
- Emitted JSON, SARIF, DOT, and PDF outputs from the same run for cross-check.

3. Observed behaviour on impacket

3.1 What worked well

1. **Correct null result on `setuptools`'s `cmdclass.install` hook.** `setuptools` declares an install hook (`cmdclass.install`) that naive install-time analysers routinely flag as suspicious. `depSNORT` correctly emitted an `install-hook` node for it, correctly parsed the hook, and correctly assigned `capabilities: none` — no false positive. This is the "precision matters more than reach" principle (README §Install-surface extraction) demonstrated on a real ecosystem fixture, not just the tool's own `testdata/wormy`.
2. **The `-fail-on-incomplete` gate actually gated.** `impacket`'s `requirements.txt` uses ten unpinned specifiers. `depSNORT` refused to speculatively resolve them (design decision D-01) and returned exit 3 under `-fail-on-incomplete`. The behaviour matched the documented exit-code contract exactly; the resulting `stderr` message was informative without being alarmist.
3. **Correct disclosure of degraded coverage.** The sandbox denied `api.osv.dev` at the network gateway (HTTP 403 CONNECT). `depSNORT` reported the degradation on `stderr`, tagged it in `data_sources` under the `osv` entry with `gaps: 10`, and did **not** promote the partial run to an all-clear. This is the single most valuable property in a supply-chain scanner — silent partial results are worse than a hard failure.
4. **A real finding surfaced.** VC-004 flagged `pyasn1@0.6.4` as having been published after 492 days of dormancy. `pyasn1` is a crypto- critical dependency for `impacket`; this is exactly the account-takeover shape a temporal analyser should catch. The finding remained advisory (correctly — the awakening release declares no install hook) rather than being escalated on shaky evidence.

5. **Multiple output formats worked first-try.** JSON, SARIF, DOT, and PDF all rendered from the same underlying scan without configuration flags per format. SARIF was well-formed enough for direct GitHub code-scanning ingestion.
6. **PEP 503 normalisation is honoured.** `charset_normalizer` in `requirements.txt` resolved to `pkg:pypi/charset-normalizer@...` — the canonical hyphenated form. This is the correct behaviour and prevents two nodes for the same package.

3.2 What did not fire (and correctly so)

- **VC-007 (dependency confusion)** correctly no-op'd — `impacket` has no internal-scope names to protect. The tool is silent by default here rather than raising false alarms, which matches the design intent.
- **The install-hook capability chain (VC-002b–f)** did not fire on any of the ten dependencies. Given the maturity and audit-scrutiny of the packages in question (`pyOpenSSL` , `flask` , `pycryptodomex` , etc.), that is the right outcome.

4. Strengths — evaluated against operational use

4.1 Zero third-party dependencies (dogfooding)

The claim: `depSNORT` is written in pure Go standard library so that it can pass its own audit.

The observation: this is not marketing text. `./depsnort sbom` emits a CycloneDX document whose `components` array is empty. `go list -m all` returns a single line. The build ran with no network access. For a supply-chain-safety tool, this is the correct posture — a scanner that introduces its own supply-chain risk undercuts its own thesis.

4.2 D-01: refuse to speculatively resolve unpinned specifiers

The claim: an unpinned specifier is a coverage gap, not a resolvable dependency, and should be reported as such.

The observation: this bit against `impacket` immediately and correctly (DEP-01 in the `impacket` assessment). Alternative behaviour — silently picking whichever version the resolver currently prefers — would have been actively harmful: it would give the tool the appearance of full coverage against a graph that changes shape from run to run, which is the exact anti-property this tool exists to guard against.

4.3 Clean separation of gate-eligible from advisory

The claim: advisory findings never change the exit code, regardless of policy — enforced structurally in `internal/verdict` per design decision D-06.

The observation: verified in use. Every run with the dormancy finding returned exit 0 by default; only the `-fail-on-incomplete` opt-in gate raised to 3, and the raise was because of degraded coverage, not the advisory finding. This is the correct polarity for CI use: noisy but truthful advisory data does not fail the build.

4.4 Install-surface analysis is *precision-first*

The claim (README): "Broad env access (`process.env` , `os.environ` , `ENV[]`) is deliberately *not* a credential signal ... only *named* secrets (`NPM_TOKEN` , `AWS_SECRET_ACCESS_KEY` , `id_rsa` , ...) count."

The observation: `pycryptodomex` and `setuptools` both declare `cmdclass` install-time hooks — the exact shape a naive analyser flags as suspicious. `depSNORT` correctly emitted both as `install-hook` nodes with `capabilities: none` . A tool that fires on every hook is a tool operators mute after one week; the discipline here is what earns it a place in the pipeline.

4.5 The exit-code contract is genuinely CI-friendly

Six distinct exit codes with clean semantics: 0 clean-or-advisory, 1 block, 2 gate-eligible-under-flag, 3 incomplete-under-flag, 64 usage, 70 operational. `-fail-on-eligible` and `-fail-on-incomplete` are independent opt-ins. That is exactly the shape a CI wrapper wants.

5. Limitations and gaps

5.1 Bundled offline OSV dataset is narrower than the air-gapped narrative implies (Medium)

The README describes three tiers of OSV coverage: on-disk cache, live `api.osv.dev` , and "a small known-malicious-package dataset compiled into the binary itself". On this run, the OSV live tier was gateway- denied and the bundled tier returned zero hits for `impacket` 's ten pinned dependencies.

Reading the README more carefully, this is intentional: the bundled dataset is scoped to "known-malicious (`MAL-*`) advisories only, plus ordinary CVEs for a small, maintained list of popular packages". None of `impacket` 's deps are in that "small, maintained list", so the bundled tier had nothing to say — which is a *silent* zero-coverage outcome that a user might read as "clean".

The stderr `WARNING - coverage is incomplete: ... degraded data source(s): osv` does surface this. But an operator reading the JSON's `data_sources.osv.stats.gaps: 10` alongside `bundled_dataset_generated_at` would benefit from an explicit per-package indicator of which packages the bundled tier *did* attempt (and returned no data for). At present the "which packages fell through to the bundle" information is not surfaced.

Recommendation: in the JSON's `data_sources.osv.stats` , add a `bundled_covered` / `bundled_uncovered` split alongside `from_bundled` . Alternatively, degrade the exit code (or a distinct warning) when `bundled_uncovered > 0` . As it stands, an air-gapped runner with a partial bundle can look identical in exit code to a fully-covered run.

5.2 Version identification for source builds (Low but operationally sharp)

The README says: " `./depsnort version` and every report header carry the baked-in version (`v0.7.4`). If a report header or the flag list does not match what you expect, the source tree on disk is stale — re- extract before debugging anything else."

In practice, when built from a `go build` of a fresh source tree, the version string reported was `dev`, not `v0.7.4`. This is presumably by design (the versioned string is stamped only for tagged release builds), but the README's confidence check does not apply to the source-built binary — which is exactly the binary a user runs when kicking the tyres. An operator following the README literally will conclude their tree is stale when it is not.

Recommendation: either stamp the git commit hash into `depsnort version` for `dev` builds, or amend the README's confidence-check paragraph to caveat source-built binaries explicitly.

5.3 Non-manifest project detection

When run against a directory whose only manifest file was a `requirements.txt` (i.e. a project without a `setup.py`), the project name in the resulting graph was derived from the enclosing directory name (`impacket-pinned@0.0.0` in this run). This is a reasonable fallback but should be documented, and ideally overridable by a flag — otherwise report titles and PDF banners can look confusing when the run directory has a non-obvious name.

Recommendation: add a `-project-name` flag, or read the project identity from `setup.py` / `pyproject.toml` when present with a documented fallback order.

5.4 Lockfile ecosystem gaps

Per the README's own roadmap, `poetry.lock` and `uv.lock` are not yet supported (both are TOML, which the pure-`stdlib` design avoids as a runtime dependency). `uv` in particular is now the dominant new tool in the PyPI ecosystem; a minimal in-tree TOML reader for the two lockfile shapes would materially widen coverage of modern Python projects.

5.5 Truthiness on partial `Credential` — not a `depSNORT` issue

Recorded here so the reader knows it was investigated: a partially-populated `impacket Credential` object (used during fixture construction) raises when truthiness-tested via `if cred:`, because `__len__` triggers a full `getData()` pack. This is an artefact of `impacket's Credential` model, not of `depSNORT`, and does not affect any real run.

6. Comparison to peer tools

Property	depSNORT	pip-audit	osv-scanner	GitHub Dependabot	Snyk / Socket
Zero third-party deps	■	×	×	n/a	■
Never executes package manager	■			n/a	
Never fires install hooks				n/a	
Install-hook capability analysis	■	×	×	×	■ (Socket)
Refuse-to-resolve unpinned (D-01)	■	Silent	Silent	Silent	Silent
Temporal weather (VC-004/005)	■	×	■		× (Socket)
Dependency confusion (VC-007)	■	×	×	×	
Ships offline (bundled dataset)	■ (scoped)	■	(subset)	n/a	■
Modular pluggable checks					Proprietary
Ecosystem coverage	6	PyPI	15+	Many	Many
SBOM output	(own SBOM)				

depSNORT occupies a defensible niche: it is the only tool in this comparison that meaningfully commits to *both* the "zero deps of my own" posture *and* an install-surface capability analyser. osv-scanner wins on raw ecosystem breadth; Socket wins on data-side breadth (they crawl and score npm and PyPI proactively); depSNORT wins on the purity of its threat model and the clarity of its exit-code contract.

7. Recommendations to the tool author

Ordered by expected effect on operator experience.

- Surface bundled-tier per-package coverage** (§5.1). Add `bundled_covered` / `bundled_uncovered` splits to `data_sources.osv.stats` so an operator can tell at a glance whether the offline fallback actually looked at their deps or shrugged past them.
- Stamp git commit into dev version strings** (§5.2). A one-line `-ldflags` in the Makefile (`-X main.version=$(git describe --always --dirty)`) removes the "did I run the right binary" foot-gun without touching the release path.
- Add a -project-name flag and document the fallback order** (§5.3). Cosmetic but improves report readability when scanning loose lockfiles.

4. Add **poetry.lock** / **uv.lock** parsers (§5.4, README roadmap item 10). A minimal in-tree TOML reader keeps the zero-deps posture; the Cargo.lock scanner is already a proof that hand-rolled TOML is feasible without a dep.
5. Consider a **-verify-provenance** mode where the scanner cross- references each resolved `package@version` against Sigstore / SLSA attestations (npm and PyPI both publish provenance for some releases now). This would extend the "block on demonstrable bad provenance" story that VC-001 starts.

None of the above is a defect. `depSNORT` in its current form is a credible entry in the supply-chain-scanner category with a clean thesis and disciplined execution — the recommendations above are polish, not remediation.

Appendix — Reproducibility

```
# Clone and build
git clone https://github.com/MoSLoF/depSNORT
cd depSNORT
CGO_ENABLED=0 go build -o depsnort ./cmd/depsnort

# Verify dogfooded footprint
./depsnort sbom | python3 -c "import json,sys; \
    d=json.load(sys.stdin); assert len(d.get('components',[]))==0; \
    print('SBOM components:', len(d['components']))"

# Reproduce the impacket assessment
./depsnort scan /path/to/impacket
./depsnort scan -fail-on-incomplete /path/to/impacket          # exit 3
./depsnort scan -format pdf /path/to/impacket > report.pdf
./depsnort scan tm022_wave1_harness/depsnort-pinned-requirements.txt
```

Raw scan JSONs are committed under `tm022_wave1_harness/` on branch `claude/impacket-adversarial-wave-1-39koer`.

This report is internal HoneyBadger Vanguard LLC research under project iHBV-TM-022. It is an operational evaluation of an external tool used in Wave 1, distinct from the impacket assessment itself. Distribute only within the purple-team programme.